Función	Tipo	Descripción	Ejemplo
filter(Predicate)	Intermedia	Filtra elementos según una condición.	edades.stream().filter(e -> e >= 18)
map(Function)	Intermedia	Transforma cada elemento en otro (ej. entero a string).	edades.stream().map(e -> "Edad: " + e)
flatMap(Function)	Intermedia	Aplana estructuras anidadas (listas dentro de listas).	listas.stream().flatMap(List::stream)
distinct()	Intermedia	Elimina elementos duplicados.	edades.stream().distinct()
sorted() / sorted(Comparator)	Intermedia	Ordena los elementos, con o sin comparador.	edades.stream().sorted() o sorted((a,b)->b-a)
limit(long)	Intermedia	Toma solo los primeros n elementos.	edades.stream().limit(3)
skip(long)	Intermedia	Omite los primeros n elementos.	edades.stream().skip(2)
peek(Consumer)	Intermedia	Ejecuta una acción por cada elemento (para depuración).	edades.stream().peek(System.out::println)
forEach(Consumer)	Terminal	Ejecuta una acción por cada elemento (sin recolectar).	edades.stream().forEach(System.out::println)
toArray()	Terminal	Convierte el stream en un arreglo.	edades.stream().toArray()
collect(Collector)	Terminal	Recolecta resultados en una colección, set, mapa, etc.	edades.stream().collect(Collectors.toList())
reduce(...)	Terminal	Reduce los elementos a un único valor acumulado.	edades.stream().reduce(0, (a,b)->a+b)
anyMatch(Predicate)	Terminal	Devuelve true si **algún** elemento cumple la condición.	edades.stream().anyMatch(e -> e >= 18)
allMatch(Predicate)	Terminal	Devuelve true si **todos** cumplen la condición.	edades.stream().allMatch(e -> e >= 18)
noneMatch(Predicate)	Terminal	Devuelve true si **ninguno** cumple la condición.	edades.stream().noneMatch(e -> e >= 18)
findFirst()	Terminal	Devuelve el **primer elemento** del stream (opcional).	edades.stream().findFirst()
findAny()	Terminal	Devuelve **cualquier elemento** (útil en streams paralelos).	edades.stream().findAny()
count()	Terminal	Devuelve el número de elementos del stream.	edades.stream().count()
max(Comparator)	Terminal	Encuentra el máximo según comparador.	edades.stream().max(Integer::compareTo)
min(Comparator)	Terminal	Encuentra el mínimo según comparador.	edades.stream().min(Integer::compareTo)

Lambda	Método abstracto	¿Qué hace?	Ejemplo con Lambda
Predicate<T>	boolean test(T t)	Evalúa una condición y devuelve true/false.	e -> e >= 18
Function<T, R>	R apply(T t)	Transforma un valor de tipo T en un R.	x -> x * 2
Consumer<T>	void accept(T t)	Ejecuta una acción, no retorna valor.	x -> System.out.println(x)
Supplier<T>	T get()	Proporciona un valor sin recibir entrada.	() -> Math.random()
BiFunction<T,U,R>	R apply(T t, U u)	Toma dos valores (T, U) y devuelve R.	(a, b) -> a + b
BiPredicate<T,U>	boolean test(T t, U u)	Evalúa condición con dos parámetros.	(a, b) -> a > b
BiConsumer<T,U>	void accept(T t, U u)	Consume dos valores y hace algo con ellos.	(a, b) -> System.out.println(a + ":" + b)
UnaryOperator<T>	T apply(T t)	Variante de Function donde entrada y salida son del mismo tipo.	x -> x * x
BinaryOperator<T>	T apply(T t1, T t2)	Variante de BiFunction con mismo tipo de entrada y salida.	(a, b) -> a * b
Runnable	void run()	No recibe parámetros, no devuelve nada.	() -> System.out.println("Hola")
Callable<V>	V call()	No recibe parámetros, devuelve un resultado.	() -> "Resultado"

Descripción
Filtrar mayores de edad en un stream.
Multi un número, convertir un objeto a String.
Imprimir elementos de una lista.
Generar un número aleatorio o instancia de objeto.
Sumar dos enteros, concatenar cadenas.
Comparar edades, validar pares clave-valor.
Imprimir un Map<K,V>.
Elevar un número a la 2
Multiplicar dos enteros, escoger máximo.
Crear hilos con Thread.
Ejecutar tareas con retorno en ExecutorService.